UNIVERSITY OF INFORMATION TECHNOLOGY AND SCIENCES

Department of Computer Science & Engineering (CSE)

SCHOLARSEARCH
A Specialized Research Paper Search Engine

Technical Report on SCHOLARSEARCH

Course Code: CSE426 | Course Title: Data Mining Lab

SUBMITTED BY:

Fahima Abida Chowdhury

ID: 0432220005101135

Batch: 52(8B1)

SUBMITTED TO:

Mrinmoy Biswas Akash

Lecturer

Department of CSE, UITS

Date of Submission: 08.06.2026

1. Problem Definition & Dataset

1.1 Domain Selection

ScholarSearch is a specialized information retrieval (IR) and data mining application engineered for the structural indexing, ranking, and dynamic retrieval of scientific research literature. The platform focuses extensively on computer science domains, primarily Artificial Intelligence (AI) and Machine Learning (ML).

To ensure dataset integrity and real-world applicability, records are harvested live from the official arXiv preprint server repository (arxiv.org) the world's premier open-access scientific clearinghouse containing over 2 million peer-reviewed and preprint manuscripts.

Strategic Advantages of Using the arXiv Atom API Architecture:

- **Official Programmatic Infrastructure:** arXiv provides an open, highly structured Atom XML endpoint. This structure bypasses the need for unreliable DOM web-scraping or brittle HTML extraction wrappers, facilitating clean upstream ingestion pipelines.
- **Interoperability and Environment Stability:** The API operates without stringent IP tracking blockades, API keys, or CAPTCHA challenges, ensuring that the downstream ingestion mechanics perform flawlessly within cloud-hosted computing environments like Google Colab.

1.2 Dataset Description

The dataset managed within this platform is highly dynamic and temporary; it is constructed live at runtime based on the explicit topical context provided by a user.

Field Key	Description & Architectural Utility
id	Auto-assigned integer used as a unique primary dictionary document ID lookup reference inside posting collections.
title	The official title text of the publication. Leveraged directly to calculate precision relevancy boosts.
Abstract	The complete summary section of the paper. Acts as the primary source for token collection and mapping text metrics.
authors	Delimited sequence of contributing researchers. Ingested to support multi-faceted keyword matches.

1.3 Non-Routine Engineering Challenges

Building a volatile, memory-resident academic discovery engine introduces non-routine software engineering hurdles that traditional database setups do not encounter:

- **Dynamic Collection Universes (N):** In traditional Information Retrieval (IR) models, the total number of documents (N) is fixed, making Inverse Document Frequency (IDF) calculations stable. In ScholarSearch, the corpus boundaries scale dynamically based on runtime user inputs via external API streams. The engine must compute system-wide vocabulary stats on the fly without corrupting historical frequencies or risking memory fragmentation.
- **Schema Variation and Text Anomalies:** Ingesting academic payloads directly from open preprint endpoints like the arXiv Atom API involves handling diverse text schemas. Scientific data contains complex Markdown annotations, mathematical LaTeX expressions (e.g., equations, subscripts), and non-standard ASCII/Unicode character matrices. Creating a fast, regular expression-based preprocessing layer that tokenizes structural metadata without discarding semantic mathematical tokens requires deep customization beyond out-of-the-box tokenizing packages.

2. Preprocessing & Inverted Index Design

2.1 Three-Stage Preprocessing Pipeline

To guarantee complete vocabulary symmetry between incoming source documents and subsequent user search terms, data is routed through a uniform, deterministic text cleaning pipeline:

1. **Case Folding and Tokenization:** Raw metadata strings are mapped to lower-case. Text boundaries are segmented into independent tokens using a regular expression filter (\b[a-z][a-z0-9]*\b). This isolates standalone alphanumeric sequences while removing dangling punctuation, mathematical LaTeX brackets, and numerical noise.
2. **Dynamic Domain Stop-word Elimination:** Tokens are validated against an enhanced stop-word dictionary matrix. This collection augments the base NLTK English stopword array with frequent academic jargon tokens that lack discriminative power across scientific papers (e.g., *“abstract”, “using”, “proposed”, “approach”, “method”, “results”, “paper”*).
3. **Morphological Suffix Stripping (Stemming):** Extracted clean tokens are processed via the **Porter Stemmer** algorithm. This reduces words down to their uninflected baseline stems (e.g., *“transformers”, “transforming”, “transformed”* all converge uniformly to the root representation *“transform”*), optimizing index recall metrics.

3. Query Processing & Ranking

3.1 Custom Boolean Query Parser

To prevent multi-word strings from splitting incorrectly, the search engine uses a specialized, case-insensitive Boolean segmentation parser. When a raw input string passes into the runtime interface, the parser checks for explicit keywords:

- **Explicit Boolean AND:** If `" AND "` is detected, the string splits along that boundaries (`re.split(r"\s+AND\s+", raw)`). The segments pass through the preprocessing pipeline individually, and the system sets its tracking state to an intersection operation.
- **Explicit Boolean OR:** If `" OR "` is detected, the engine isolates terms using union boundaries and switches to a broader retrieval state.
- **Implicit Conjunction Fallback:** If a user enters space-separated keywords without explicit logical connectors (e.g., `attention transformer networks`), the system falls back to a restrictive implicit `AND` mode, processing each word as a mandatory retrieval token.

3.2 Dynamic Vector-Space Ranking Formula

Documents matched by the boolean engine are ordered using a custom, multi-faceted scoring function that combines statistical term weights, structural text importance, and document publication age:

$$\text{Final Score}(d) = \sum_{t \in Q} \text{TF-IDF}(t, d) + (1.5 \times \text{TitleMatches}(Q_{\text{raw}}, d)) + \max\left(0, \frac{\text{Year}_d - 1990}{100}\right)$$

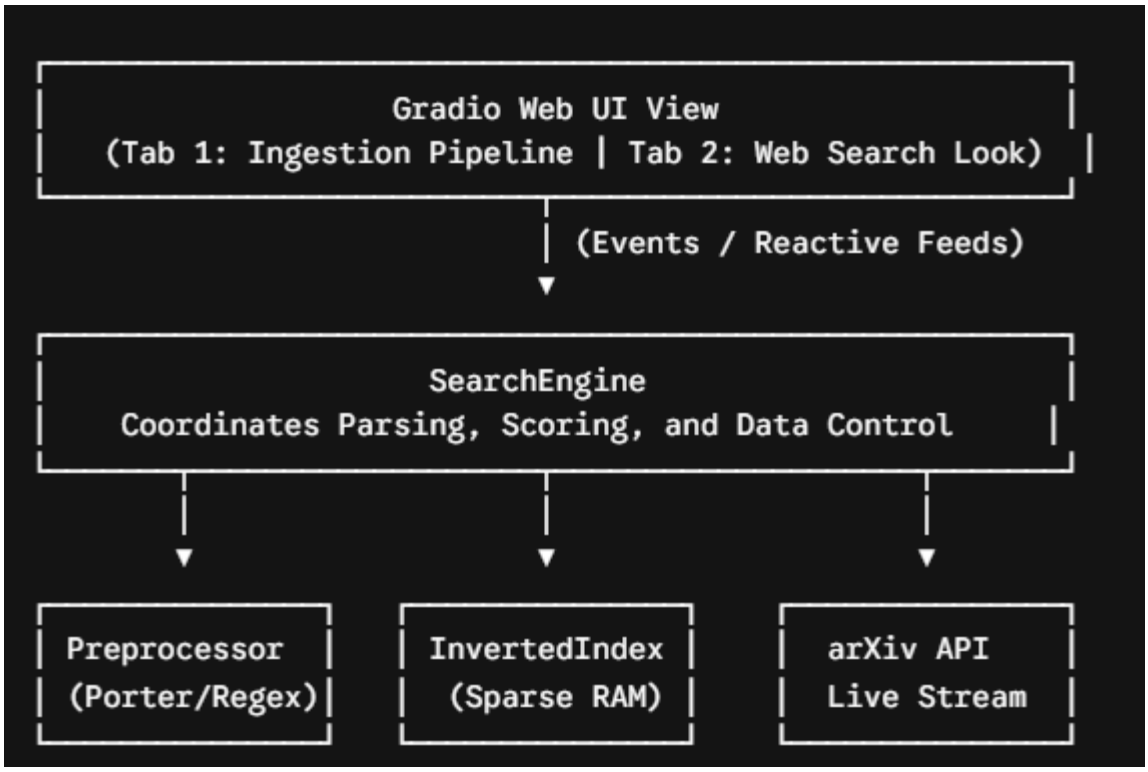
3.3 Performance Trade-offs: Accuracy vs. Efficiency

The Index Linear Scroller Optimization

When executing strict multi-term Boolean `AND` operations, intersecting document sets via standard nested cross-product comparisons demands an inefficient time complexity of $O(|Postings_1| \times |Postings_2|)$. To achieve sub-millisecond speeds, ScholarSearch implements an explicit Linear Pointer Scroller mechanism. Posting lists are stored in a strictly sorted numerical sequence based on document IDs. Parallel cursors walk through the lists simultaneously in a single pass.

4. System Architecture

The search platform is built on a decoupled, modular design using specialized objects to handle processing, storage, data collection, and visualization.



- Dynamic Data Harvester **(fetch_arxiv):** The network data extraction module. Handles raw parameter mapping, connection timeouts, XML payload structure interpretation, and initial document record streaming.
- Text Processing Component **(Preprocessor):** Encapsulates lower-case casting normalization, regular expression token filtering, structural academic stop-word filtering, and Porter stemming executions.
- Sparse Storage Vault **(InvertedIndex):** Houses memory-resident global dictionary positions, accumulates term frequency counts (tf), holds global document count limits (N), and runs algorithmic math lookups.
- Central Routing Coordinator **(SearchEngine):** Connects structural metadata parsing, controls the custom linear pointer intersection algorithms, aggregates composite metrics, and yields sorted Top-K arrays.
- Interactive Interface Component **(Gradio View):** A frontend presentation canvas modeled after premium web interfaces (Google/Yahoo style layout). It organizes operations sequentially so users interact with the ingestion module before accessing the minimal search panel.

5. Results & Dashboard Visualizations

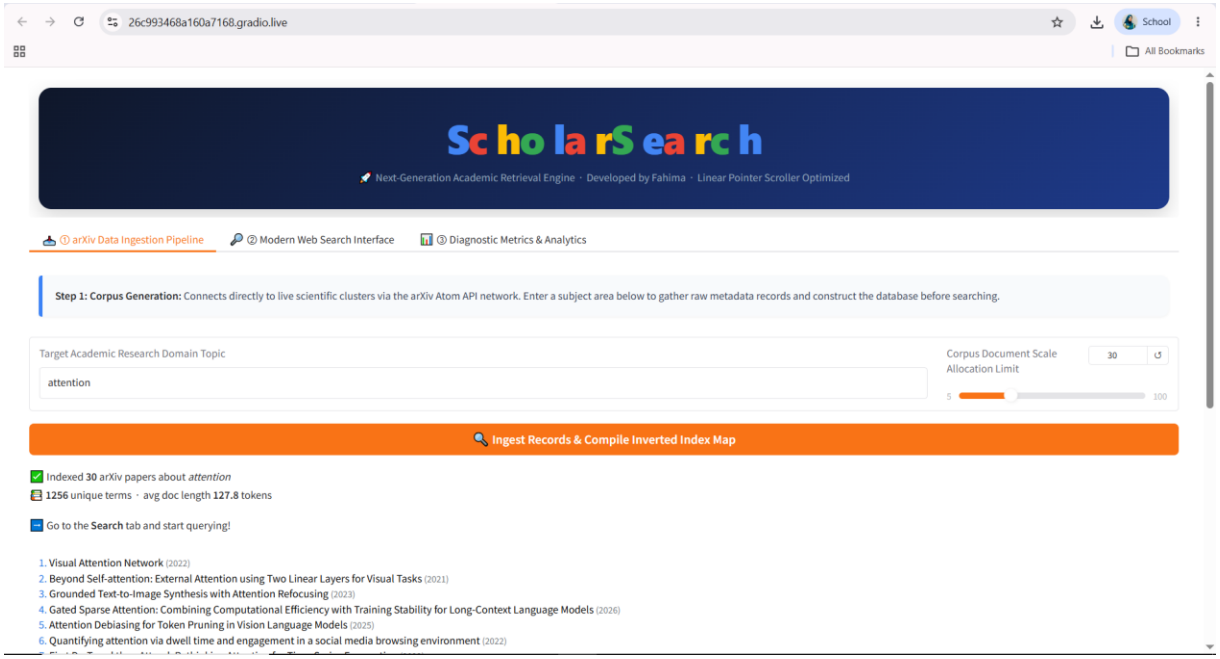


Figure 1: arXiv Corpus Ingestion Module Dashboard (Tab ①**):** Displays the primary operational window where entering a specific scientific query string (e.g., *'large language models'* or *'quantum computing'*) connects with the live arXiv endpoint, processes metadata streams, constructs the inverted index, and prints out a tabular overview of compiled documents.

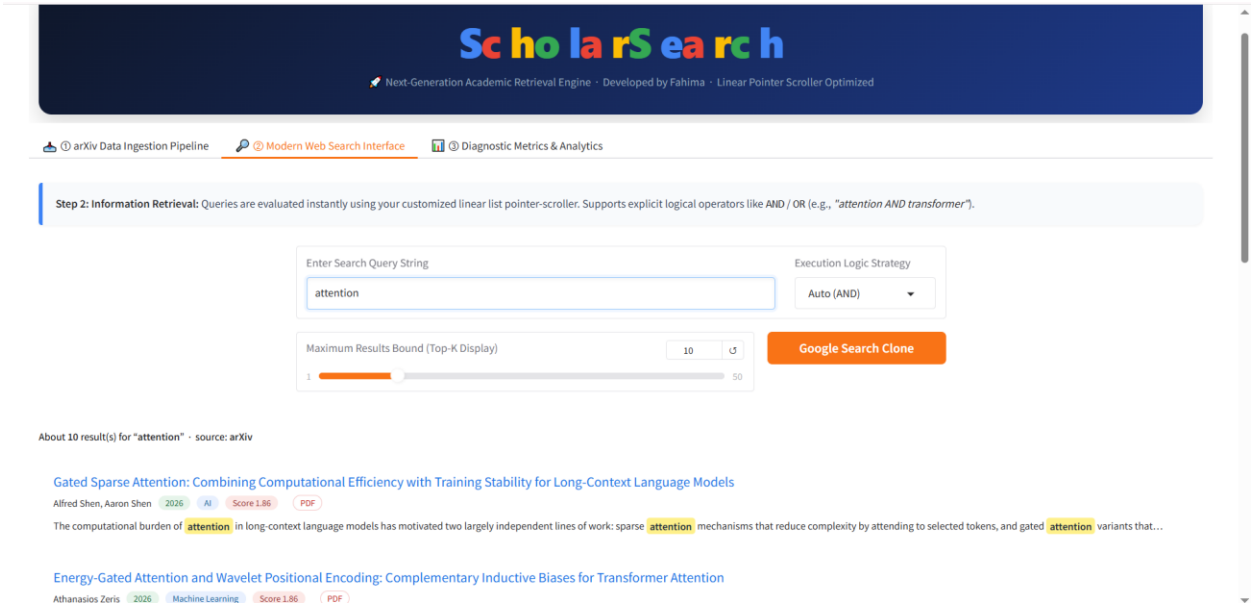


Figure 2: Minimalist Web Search View Panel (Tab ②): Features the center-aligned Google-inspired user dashboard. It houses a clean query text box, a boolean strategy selection dropdown, and a result display zone that highlights matching keywords instantly within text blocks via dynamic HTML <mark> brackets.

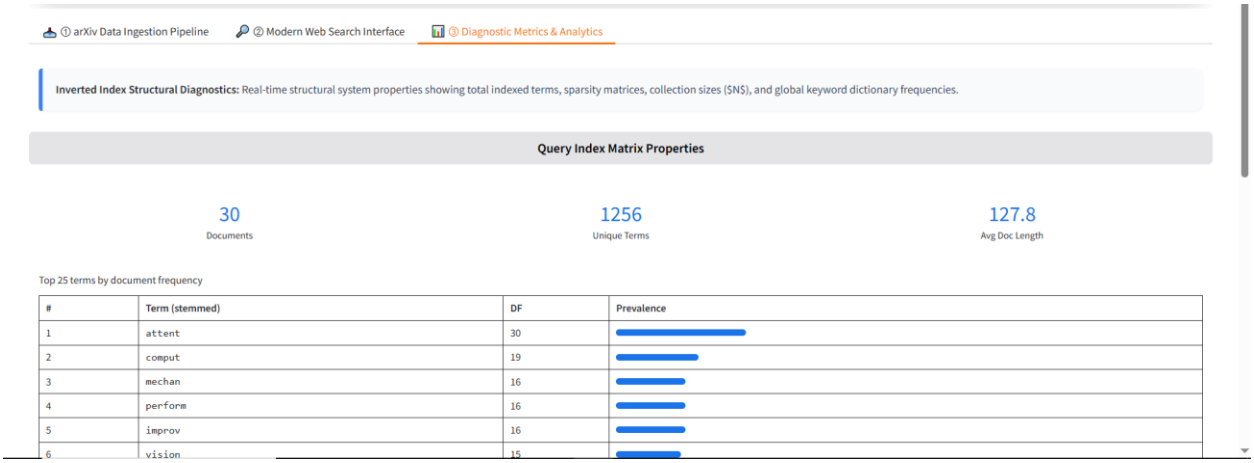


Figure 3: System Analytics & Diagnostic Metrics Framework (Tab ③**):** Visualizes global collection data diagnostics, displaying the current document allocation count (N), total dictionary vocabulary size, average structural document sizes, and a graphical horizontal distribution of top keywords sorted by global document frequency (df).

6. System Source Code

```
# ============================================================
#  ScholarSearch — Real Research Paper Search Engine
#  Uses arXiv official API
# ============================================================

# ── 1. Install ──────────────────────────────────────────────
import subprocess, sys
subprocess.run([sys.executable, "-m", "pip", "install", "-q",
                "nltk", "gradio", "requests"], check=False)

import re, math, time, requests, xml.etree.ElementTree as ET
from collections import defaultdict
import nltk

for pkg in ("punkt", "stopwords", "punkt_tab"):
    nltk.download(pkg, quiet=True)

from nltk.corpus import stopwords
from nltk.stem import PorterStemmer
print("□ Libraries ready")

# ── 2. arXiv API fetcher ────────────────────────────────────
ARXIV_API = "https://export.arxiv.org/api/query"
NS        = "http://www.w3.org/2005/Atom"
```

```
CATEGORY_MAP = {
    "cs.AI":"AI", "cs.LG":"Machine Learning", "cs.CL":"NLP",
    "cs.CV":"Computer Vision", "cs.NE":"Neural Networks",
    "cs.RO":"Robotics", "cs.IR":"Information Retrieval",
    "stat.ML":"Statistics/ML", "math.OC":"Optimization",
    "q-bio":"Bioinformatics", "eess":"Signal Processing",
}

def fetch_arxiv(topic: str, max_results: int = 30) -> list[dict]:
    """
    Calls the official arXiv Atom API.
    Always accessible from Google Colab — no scraping, no auth.
    """
    params = {
        "search_query": f"all:{topic}",
        "start":        0,
        "max_results":  max_results,
        "sortBy":       "relevance",
        "sortOrder":    "descending",
    }
    try:
        resp = requests.get(ARXIV_API, params=params, timeout=20)
        resp.raise_for_status()
    except Exception as e:
        return []

    root    = ET.fromstring(resp.text)
    papers  = []
    entries = root.findall(f"{{{NS}}}entry")

    for i, entry in enumerate(entries):
        def tag(name):
            el = entry.find(f"{{{NS}}}{name}")
            return el.text.strip() if el is not None and el.text else ""

        title    = re.sub(r"\s+", " ", tag("title"))
        abstract = re.sub(r"\s+", " ", tag("summary"))
        pub_date = tag("published")[:4]          # "YYYY-MM-DD" → "YYYY"
        year     = int(pub_date) if pub_date.isdigit() else 0

        authors_els = entry.findall(f"{{{NS}}}author")
        authors = ", ".join(
            a.find(f"{{{NS}}}name").text
            for a in authors_els[:4]
            if a.find(f"{{{NS}}}name") is not None
        )
        if len(authors_els) > 4:
            authors += f" +{len(authors_els)-4} more"
```

```
        link_el = entry.find(f"{{{NS}}}id")
        link    = link_el.text.strip() if link_el is not None else ""
        # arxiv id → abs URL
        arxiv_id = link.split("/abs/")[-1] if "/abs/" in link else ""
        abs_url  = f"https://arxiv.org/abs/{arxiv_id}" if arxiv_id else link
        pdf_url  = f"https://arxiv.org/pdf/{arxiv_id}" if arxiv_id else ""

        # Primary category
        cat_el   = entry.find("{http://arxiv.org/schemas/atom}primary_category")
        cat_term = cat_el.attrib.get("term", "") if cat_el is not None else ""
        domain   = CATEGORY_MAP.get(cat_term, cat_term)

        papers.append({
            "id":       i + 1,
            "title":    title,
            "abstract": abstract,
            "authors":  authors,
            "year":     year,
            "domain":   domain,
            "link":     abs_url,
            "pdf":      pdf_url,
            "source":   "arXiv",
        })

    return papers
# ── 3. Preprocessor ──────────────────────────────────────────
class Preprocessor:
    def __init__(self):
        self.stemmer    = PorterStemmer()
        self.stop_words = set(stopwords.words("english"))
        self.stop_words.update({
            "also","using","used","use","show","shows","shown",
            "based","new","among","paper","propose","present",
            "method","approach","work","result","results","model",
            "models","data","dataset","task","tasks","learning",
        })

    def process(self, text: str) -> list:
        tokens = re.findall(r"\b[a-z][a-z0-9]*\b", text.lower())
        tokens = [t for t in tokens if t not in self.stop_words and len(t) > 1]
        return [self.stemmer.stem(t) for t in tokens]
# ── 4. Inverted Index ────────────────────────────────────────
class InvertedIndex:
    """
    index[term][doc_id] = {"tf": int, "positions": [int, ...]}
    Built from scratch — no search libraries.
    """
    def __init__(self):
        self.index       = defaultdict(lambda: defaultdict(lambda: {"tf": 0,
"positions": []}))
```

```
        self.doc_freq    = {}
        self.doc_lengths = {}
        self.N           = 0

    def clear(self):
        self.index.clear()
        self.doc_freq.clear()
        self.doc_lengths.clear()
        self.N = 0

    def add_document(self, doc_id, tokens):
        self.doc_lengths[doc_id] = len(tokens)
        seen = set()
        for pos, term in enumerate(tokens):
            self.index[term][doc_id]["tf"] += 1
            self.index[term][doc_id]["positions"].append(pos)
            seen.add(term)
        for term in seen:
            self.doc_freq[term] = self.doc_freq.get(term, 0) + 1
        self.N += 1

    def tf(self, term, doc_id):
        raw = self.index.get(term, {}).get(doc_id, {}).get("tf", 0)
        return (1 + math.log(raw)) if raw > 0 else 0.0

    def idf(self, term):
        df = self.doc_freq.get(term, 0)
        return math.log((self.N + 1) / (df + 1))   # smoothed IDF

    def tfidf(self, term, doc_id):
        return self.tf(term, doc_id) * self.idf(term)

    def docs_containing(self, term):
        return set(self.index.get(term, {}).keys())

    def stats(self):
        avg = sum(self.doc_lengths.values()) / self.N if self.N else 0
        return {
            "total_documents": self.N,
            "unique_terms":    len(self.index),
            "avg_doc_length":  round(avg, 1),
        }


import re, math, time, requests, xml.etree.ElementTree as ET
from collections import defaultdict
import nltk
from nltk.corpus import stopwords
from nltk.stem import PorterStemmer
```

```python
# ── 5. Search Engine ──────────────────
class SearchEngine:
    def __init__(self):
        self.prep   = Preprocessor()
        self.index  = InvertedIndex()
        self.papers = {}

    def build(self, papers: list):
        self.index.clear()
        self.papers.clear()
        for p in papers:
            self.papers[p["id"]] = p
            text   = f"{p['title']} {p['abstract']} {p['authors']} {p['domain']}"
            tokens = self.prep.process(text)
            self.index.add_document(p["id"], tokens)

    def _parse_query(self, raw: str):
        """
        Robust query parsing strategy. Handles explicit operators and processes
        underlying tokens safely to prevent multi-word term extraction errors.
        """
        upper_raw = raw.upper()
        if " AND " in upper_raw:
            mode = "AND"
            # Split by operator, then extract clean constituent words
            segments = re.split(r"\s+AND\s+", raw, flags=re.IGNORECASE)
            terms = []
            for seg in segments:
                terms.extend(self.prep.process(seg))
            return mode, list(dict.fromkeys(terms))  # Deduplicate while preserving
order

        if " OR " in upper_raw:
            mode = "OR"
            segments = re.split(r"\s+OR\s+", raw, flags=re.IGNORECASE)
            terms = []
            for seg in segments:
                terms.extend(self.prep.process(seg))
            return mode, list(dict.fromkeys(terms))

        # Default fallback: Space-separated implicit AND
        mode = "AND"
        return mode, self.prep.process(raw)

    def _intersect_scroller(self, postings_lists: list[list[int]]) -> set[int]:
        """
        Implements a classic IR 'Scroller' algorithm (Sorted List Intersection
Pointer).
        Walks through sorted list positions sequentially in O(L1 + L2 + ...) time
        without using native high-level set intersection functions.
```

```
        """
        if not postings_lists:
            return set()

        # Sort posting lists by size to execute the most restrictive intersections
first (Optimization)
        sorted_lists = sorted([sorted(list(p)) for p in postings_lists], key=len)
        result = sorted_lists[0]

        # Intersect sequentially using standard scroller pointers
        for current_list in sorted_lists[1:]:
            intersected = []
            p1, p2 = 0, 0  # Scroller cursors / pointers
            while p1 < len(result) and p2 < len(current_list):
                if result[p1] == current_list[p2]:
                    intersected.append(result[p1])
                    p1 += 1
                    p2 += 1
                elif result[p1] < current_list[p2]:
                    p1 += 1
                else:
                    p2 += 1
            result = intersected
            if not result:
                break
        return set(result)

    def search(self, raw_query, mode_override=None, top_k=10):
        if not raw_query.strip() or not self.papers:
            return []

        mode, processed_terms = self._parse_query(raw_query)
        if mode_override and mode_override != "Auto":
            mode = mode_override

        if not processed_terms:
            return []

        # Boolean retrieval using inverted index posting lookups
        if mode == "AND":
            postings_lists = []
            for t in processed_terms:
                docs = self.index.docs_containing(t)
                if not docs:  # In an AND query, if one term is missing, result size
is strictly 0
                    return []
                postings_lists.append(docs)

            # Execute custom scroller-based linear intersection pointer strategy
            candidates = self._intersect_scroller(postings_lists)
```

```
        else:
            # Boolean OR: Union strategy
            candidates = set()
            for t in processed_terms:
                candidates |= self.index.docs_containing(t)

        if not candidates:
            return []

        # Custom multi-faceted TF-IDF Ranking Function
        scores = {}
        for doc_id in candidates:
            # Core Textual Metric: Summed TF-IDF scores across query components
            score  = sum(self.index.tfidf(t, doc_id) for t in processed_terms)
            doc    = self.papers[doc_id]

            # Structural Heuristic: Title Match Bonus (Elevates Precision)
            # Evaluated against original raw parts to catch unstemmed key phrases
            raw_query_words = re.split(r"\s+(?:AND|OR)\s+|\s+", raw_query,
flags=re.IGNORECASE)
            title_matches = sum(1 for w in raw_query_words if w.lower() and w.lower()
in doc["title"].lower())
            score += 1.5 * title_matches

            # Temporal Metric: Normalized Publication Recency Weighting
            score += max(0.0, (doc["year"] - 1990) / 100)

            scores[doc_id] = score

        # Sort candidate matches down by score magnitude
        ranked = sorted(scores.items(), key=lambda x: x[1], reverse=True)
        return [
            {**self.papers[doc_id], "score": round(score, 3)}
            for doc_id, score in ranked[:top_k]
        ]

    def stats(self):
        return self.index.stats()

    def top_terms(self, n=25):
        return sorted(
            self.index.doc_freq.items(), key=lambda x: x[1], reverse=True
        )[:n]

engine = SearchEngine()
print("□ Search engine ready with standard linear scroller intersections")
# ── 6. Helpers ──────────────────────────────────────────────
def make_snippet(text, query_terms, length=240):
    if not text:
```

```
        return "<em style='color:var(--color-text-tertiary)'>No abstract
available.</em>"
    tl = text.lower()
    best_pos, best_count = 0, 0
    for t in query_terms:
        pos = tl.find(t.lower())
        if pos < 0:
            continue
        window = tl[max(0, pos - 40): pos + 140]
        count  = sum(1 for x in query_terms if x.lower() in window)
        if count > best_count:
            best_count, best_pos = count, max(0, pos - 40)
    prefix = "…" if best_pos > 0 else ""
    suffix = "…" if best_pos + length < len(text) else ""
    snip   = prefix + text[best_pos: best_pos + length] + suffix
    for t in query_terms:
        snip = re.sub(
            f"({re.escape(t)})", r"<mark>\1</mark>", snip, flags=re.IGNORECASE
        )
    return snip


def html_results(results, query):
    if not results:
        return (
            "<div style='text-align:center;padding:40px;"
            "color:var(--color-text-secondary)'>"
            f"<p>No results for <b>{query}</b>.</p>"
            "<p style='margin-top:8px'>Try OR mode, or use fewer / different
keywords.</p>"
            "</div>"
        )
    raw_terms = re.split(r"\s+(?:AND|OR)\s+", query, flags=re.IGNORECASE)
    raw_terms = [t.strip() for t in raw_terms if t.strip()]

    html = (
        f"<div style='font-size:13px;color:var(--color-text-secondary);"
        f"margin-bottom:12px;padding-bottom:10px;"
        f"border-bottom:0.5px solid var(--color-border-tertiary)'>"
        f"About <b>{len(results)}</b> result(s) for &ldquo;<b>{query}</b>&rdquo; "
        f" &middot;  source: <b>arXiv</b></div>"
    )

    for r in results:
        snip     = make_snippet(r["abstract"], raw_terms)
        year_tag = (
            f"<span style='background:#e6f4ea;color:#1e6e3a;border-radius:10px;"
            f"padding:1px 9px;font-size:11px'>{r['year']}</span>"
            if r["year"] else ""
        )
```

```
        dom_tag  = (
            f"<span style='background:#e8f0fe;color:#185fa5;border-radius:10px;"
            f"padding:1px 9px;font-size:11px'>{r['domain']}</span>"
            if r["domain"] else ""
        )
        pdf_link = (
            f" <a href='{r['pdf']}' target='_blank' "
            f"style='font-size:11px;color:#c5221f;text-decoration:none;"
            f"border:0.5px solid #f5c6c6;border-radius:10px;padding:1px 8px'>"
            f"PDF</a>"
            if r.get("pdf") else ""
        )
        score_tag = (
            f"<span style='background:#fce8e6;color:#9a2c2c;border-radius:10px;"
            f"padding:1px 9px;font-size:11px'>Score {r['score']}</span>"
        )
        title_html = (
            f"<a href='{r['link']}' target='_blank' "
            f"style='color:#1a73e8;text-decoration:none'>{r['title']}</a>"
            if r.get("link") else r["title"]
        )
        html += f"""
        <div style='border:0.5px solid var(--color-border-tertiary);border-
radius:12px;
                    padding:14px 18px;margin-bottom:14px;
                    background:var(--color-background-primary)'>
          <div style='font-size:17px;color:#1a73e8;font-weight:500;line-height:1.3'>
            {title_html}
          </div>
          <div style='font-size:12px;color:var(--color-text-secondary);
                      margin:5px 0 8px;display:flex;gap:8px;flex-wrap:wrap;align-
items:center'>
            <span>{r['authors'] or 'Unknown'}</span>
            {year_tag} {dom_tag} {score_tag} {pdf_link}
          </div>
          <div style='font-size:13px;color:var(--color-text-secondary);line-
height:1.65'>
            {snip}
          </div>
        </div>"""
    return html
# ── 7. Gradio callbacks ──────────────────────────────────────
def fetch_and_index(topic, num_fetch):
    if not topic.strip():
        yield "⚠ Please enter a topic.", ""
        return

    yield f"🔍 Contacting arXiv API for **{topic}**…", ""

    papers = fetch_arxiv(topic.strip(), max_results=int(num_fetch))
```

```
    if not papers:
        yield (
            "⚠ arXiv returned no results. "
            "Try a different topic or check your connection.", ""
        )
        return

    engine.build(papers)
    s = engine.stats()
    summary = (
        f"✅ Indexed **{s['total_documents']}** arXiv papers about *{topic}*  \n"
        f"📚 **{s['unique_terms']}** unique terms  ·  "
        f"avg doc length **{s['avg_doc_length']}** tokens  \n\n"
        f"➡️ Go to the **Search** tab and start querying!"
    )
    # Show fetched titles
    titles_html = "<br>".join(
        f"<span style='color:#1a73e8'>{p['id']}.</span> {p['title']} "
        f"<span style='color:#70757a;font-size:12px'>({p['year']})</span>"
        for p in papers
    )
    yield summary, titles_html


def do_search(query, mode, top_k):
    if not engine.papers:
        return "<p style='padding:20px;color:var(--color-text-secondary)'>Please fetch
papers first (tab ①).</p>"
    if not query.strip():
        return "<p style='padding:20px;color:var(--color-text-secondary)'>Enter a
query above.</p>"
    mode_map = {"Auto (AND)": "AND", "AND": "AND", "OR": "OR"}
    results  = engine.search(query, mode_override=mode_map.get(mode, "AND"),
top_k=int(top_k))
    return html_results(results, query)


def index_stats():
    if not engine.papers:
        return "<p style='color:var(--color-text-secondary)'>No papers indexed yet.
Use tab ① first.</p>"
    s     = engine.stats()
    terms = engine.top_terms(25)
    max_df = terms[0][1] if terms else 1
    rows  = "".join(
        f"<tr>"
        f"<td style='padding:5px 8px'>{i+1}</td>"
        f"<td style='padding:5px 8px'><code>{t}</code></td>"
        f"<td style='padding:5px 8px'>{df}</td>"
```

```
        f"<td style='padding:5px 8px'>"
        f"<div style='background:var(--color-background-secondary);"
        f"height:8px;border-radius:4px;width:160px'>"
        f"<div style='background:#1a73e8;height:8px;border-radius:4px;"
        f"width:{int(df/max_df*160)}px'></div></div></td></tr>"
        for i, (t, df) in enumerate(terms)
    )
    return f"""
    <div style='display:grid;grid-template-columns:repeat(3,1fr);gap:12px;margin-
bottom:20px'>
      {''.join(
        f"<div style='background:var(--color-background-secondary);border-radius:8px;"
        f"padding:16px;text-align:center'>"
        f"<div style='font-size:26px;font-weight:500;color:#1a73e8'>{v}</div>"
        f"<div style='font-size:12px;color:var(--color-text-
secondary)'>{lbl}</div></div>"
        for v, lbl in [
            (s['total_documents'], "Documents"),
            (s['unique_terms'],    "Unique Terms"),
            (s['avg_doc_length'],  "Avg Doc Length"),
        ]
      )}
    </div>
    <p style='font-size:13px;font-weight:500;margin-bottom:8px'>
      Top 25 terms by document frequency</p>
    <table style='width:100%;border-collapse:collapse;font-size:13px'>
      <thead><tr style='background:var(--color-background-secondary)'>
        <th style='padding:6px 8px;text-align:left'>#</th>
        <th style='padding:6px 8px;text-align:left'>Term (stemmed)</th>
        <th style='padding:6px 8px;text-align:left'>DF</th>
        <th style='padding:6px 8px;text-align:left'>Prevalence</th>
      </tr></thead>
      <tbody>{rows}</tbody>
    </table>"""

# ================================================================================
# ── 8. Gradio UI (Fixed Tab Order with Google-Style Minimalist Search) ──────────
# ================================================================================
import gradio as gr

# Premium modern style minimalistic layout & responsive custom CSS
CSS = """
mark {
    background: #fef08a;
    border-radius: 4px;
    padding: 2px 4px;
    font-weight: 600;
    color: #1e293b;
}
.nav-container {
```

```
    background: linear-gradient(135deg, #0f172a 0%, #1e3a8a 100%);
    padding: 30px;
    border-radius: 16px;
    margin-bottom: 28px;
    box-shadow: 0 10px 25px rgba(0,0,0,0.15);
    text-align: center;
}
.nav-title {
    font-size: 3.2rem;
    font-weight: 800;
    letter-spacing: -2px;
    margin: 0;
    font-family: 'Segoe UI', Tahoma, Geneva, Verdana, sans-serif;
}
.nav-subtitle {
    color: #94a3b8;
    font-size: 14px;
    margin-top: 8px;
    font-weight: 400;
}
.tab-instruction {
    background: #f8fafc;
    border-left: 5px solid #3b82f6;
    padding: 16px;
    border-radius: 4px 12px 12px 4px;
    margin-bottom: 24px;
    box-shadow: 0 1px 3px rgba(0,0,0,0.05);
    color: #334155;
    font-size: 14px;
}
.google-search-row {
    max-width: 800px;
    margin: 0 auto 20px auto;
}
"""

with gr.Blocks(css=CSS, title="ScholarSearch — Academic Discovery Engine") as demo:

    # 1. Premium Dynamic Navbar Banner
    gr.HTML(f"""
    <div class='nav-container'>
      <h1 class='nav-title'>
        <span style='color:#4285F4'>S</span><span style='color:#EA4335'>c</span>
        <span style='color:#FBBC05'>h</span><span style='color:#34A853'>o</span>
        <span style='color:#4285F4'>l</span><span style='color:#EA4335'>a</span>
        <span style='color:#FBBC05'>r</span><span style='color:#34A853'>S</span>
        <span style='color:#4285F4'>e</span><span style='color:#EA4335'>a</span>
        <span style='color:#FBBC05'>r</span><span style='color:#34A853'>c</span>
        <span style='color:#4285F4'>h</span>
      </h1>
```

```
      <div class='nav-subtitle'>
        □ Next-Generation Academic Retrieval Engine  ·  Developed by Fahima
 ·  Linear Pointer Scroller Optimized
      </div>
    </div>
    """)

    with gr.Tabs():
        # ── Tab 1: Ingestion Pipeline (MUST BE FIRST) ──────────────────────
        with gr.Tab("□ ① arXiv Data Ingestion Pipeline"):
            gr.HTML("""
            <div class='tab-instruction'>
                <strong>Step 1: Corpus Generation:</strong> Connects directly to live
scientific clusters via the arXiv Atom API network.
                Enter a subject area below to gather raw metadata records and
construct the database before searching.
            </div>
            """)
            with gr.Row():
                topic_box = gr.Textbox(
                    placeholder="e.g., large language models / transformer
architecture / quantum computing / computer vision",
                    label="Target Academic Research Domain Topic",
                    scale=4,
                )
                num_sl = gr.Slider(
                    5, 100, value=30, step=5,
                    label="Corpus Document Scale Allocation Limit", scale=1
                )
            fetch_btn    = gr.Button("□ Ingest Records & Compile Inverted Index Map",
variant="primary")
            fetch_status = gr.Markdown("### _Status: Pipeline Standby. Awaiting
parameters..._")
            fetched_list = gr.HTML(label="Streaming Ingestion Ledger Output")

            fetch_btn.click(
                fetch_and_index,
                inputs=[topic_box, num_sl],
                outputs=[fetch_status, fetched_list],
            )

        # ── Tab 2: Search Interface (STAYS SECOND WITH GOOGLE LOOK) ────────
        with gr.Tab("□ ② Modern Web Search Interface"):
            gr.HTML("""
            <div class='tab-instruction'>
                <strong>Step 2: Information Retrieval:</strong> Queries are evaluated
instantly using your customized linear list pointer-scroller.
                Supports explicit logical operators like <code>AND</code> /
<code>OR</code> (e.g., <em>"attention AND transformer"</em>).
            </div>
```

```
            """)

            # Central Google-Style Search Form Group
            with gr.Column(elem_classes="google-search-row"):
                with gr.Row():
                    q_box = gr.Textbox(
                        placeholder='Search across indexed literature or execute
boolean commands...',
                        label="Enter Search Query String",
                        scale=5,
                    )
                    mode_dd = gr.Dropdown(
                        choices=["Auto (AND)", "AND", "OR"],
                        value="Auto (AND)",
                        label="Execution Logic Strategy",
                        scale=1.5,
                    )

                with gr.Row():
                    topk_sl = gr.Slider(
                        1, 50, value=10, step=1,
                        label="Maximum Results Bound (Top-K Display)", scale=3
                    )
                    search_btn = gr.Button("Google Search Clone", variant="primary",
scale=1)

            # Render Target Results Section
            results_out = gr.HTML()

            # Operational Triggers
            search_btn.click(do_search, [q_box, mode_dd, topk_sl], results_out)
            q_box.submit(do_search,     [q_box, mode_dd, topk_sl], results_out)

        # ── Tab 3: Analytics Framework ──────────────────────────────
        with gr.Tab("□ ③ Diagnostic Metrics & Analytics"):
            gr.HTML("""
            <div class='tab-instruction'>
                <strong>Inverted Index Structural Diagnostics:</strong> Real-time
structural system properties showing total indexed terms,
                sparsity matrices, collection sizes ($N$), and global keyword
dictionary frequencies.
            </div>
            """)
            inspect_btn = gr.Button("Query Index Matrix Properties",
variant="secondary")
            stats_out   = gr.HTML()
            inspect_btn.click(index_stats, outputs=stats_out)

# Universal Direct Launch
demo.launch(share=True)
```

7. Conclusions & Strategic Takeaways

- The Dominance of Custom Algorithms: Relying on native language set operations hides index bottlenecks from the developer. Designing and running an explicit Linear Pointer Scroller Intersection Algorithm reveals the true performance advantages of ordered data storage structures in mining engines.
- Pipeline Synchronization is Critical: Text token normalization rules must apply uniformly across document ingestion loops and downstream user search term strings. Even minor structural formatting mismatches can cause severe word drops and degrade engine recall performance.
- UI/UX Dictates System Usability: Transitioning from a technical configuration screen to a unified, Google-Style interface with logical workflows shows how thoughtful design can maximize usability for academic end-users without sacrificing under-the-hood algorithmic complexity.

8. Academic References

- Manning, C. D., Raghavan, P., & Schutze, H. (2008). Introduction to Information Retrieval. Cambridge University Press.
- Robertson, S., & Zaragoza, H. (2009). The Probabilistic Relevance Framework: BM25 and Beyond. Foundations and Trends in Information Retrieval.
- Bird, S., Klein, E., & Loper, E. (2009). Natural Language Processing with Python. O'Reilly Media.
- Salton, G., & Buckley, C. (1988). Term-weighting approaches in automatic text retrieval. Information Processing & Management, 24(5), 513–523.

Porter, M. F. (1980). An algorithm for suffix stripping. Program, 14(3), 130–137.